



Plateforme de Recrutement Intelligente

Rapport Technique Complet

Version 1.0 — Mai 2026

Domaines couverts

Architecture Système • API REST • Intelligence
Artificielle
Base de Données • Sécurité • Diagrammes de
Séquence

Équipe Tanit Talent

Tunisie — 2026

Compilé avec — Mai 2026

Table des matières

1	Présentation du Projet	4
1.1	Vue d'ensemble	4
1.2	Périmètre et Rôles	5
2	Architecture Système	6
2.1	Vue Architecturale Globale	6
2.2	Stack Technologique	7
3	Diagrammes de Séquence	8
3.1	Connexion (Login)	8
3.2	Inscription	9
3.3	Upload CV et Analyse IA	10
3.4	Candidature à une offre	11
3.5	Publication d'une offre (Recruteur)	11
3.6	Chat avec l'assistant Tanit AI	12
3.7	Suggestions d'offres personnalisées	12
3.8	Mise à jour du statut d'une candidature	13
3.9	Récapitulatif des flux	14
4	Modèle de Données	15
4.1	Schéma Entité-Relation	15
4.2	Schémas Prisma principaux	15
5	API REST — Documentation	17
5.1	Authentification (/api/auth)	17
5.2	Offres d'emploi (/api/jobs)	17
5.3	Candidatures (/api/applications)	18
5.4	Proxy IA (/api/ai)	18
6	Sécurité et Authentification	19
6.1	Mécanisme JWT avec cookie HttpOnly	19
6.2	Mesures de Sécurité	20

7 Déploiement et Infrastructure	21
7.1 Docker Compose	21
7.2 Variables d'Environnement	22
Conclusion	23
A Glossaire	24

Table des figures

2.1	Architecture globale de Tanit Talent	6
3.1	Diagramme de séquence — Connexion	8
3.2	Diagramme de séquence — Inscription	9
3.3	Diagramme de séquence — Upload CV et Analyse IA	10
3.4	Diagramme de séquence — Candidature à une offre	11
3.5	Diagramme de séquence — Publication d'une offre	11
3.6	Diagramme de séquence — Chat Tanit AI	12
3.7	Diagramme de séquence — Suggestions d'offres	12
3.8	Diagramme de séquence — Mise à jour du statut d'une candidature . . .	13
4.1	Diagramme Entité-Relation simplifié	15
6.1	Flux d'authentification JWT avec cookie HttpOnly	19

Chapitre 1

Présentation du Projet

1.1 Vue d'ensemble

À propos de Tanit Talent

Tanit Talent est une application web de type *job board* nouvelle génération conçue pour la mise en relation entre candidats, recruteurs et administrateurs. La plateforme intègre nativement un microservice d'Intelligence Artificielle (FastAPI) capable d'analyser les CVs, de scorer les candidatures et de proposer des offres personnalisées.

La plateforme repose sur trois piliers fondamentaux :

1. **Accessibilité** : Interface web responsive accessible depuis tout navigateur moderne.
2. **Intelligence** : Un moteur IA intégré qui améliore l'expérience candidat et recruteur.
3. **Modularité** : Architecture microservices permettant une évolution indépendante des composants.

1.2 Périmètre et Rôles

Table 1.1 — *Rôles et permissions dans Tanit Talent*

Rôle	Accès	Fonctionnalités clés
CANDIDATE	Tableau de bord, profil, CV	Upload CV, score IA, suggestions, candidatures
EMPLOYER	Espace recruteur, offres	Publication offres, consultation candidatures
ADMIN	Toutes les routes	Gestion utilisateurs, suppression comptes

Chapitre 2

Architecture Système

2.1 Vue Architecturale Globale

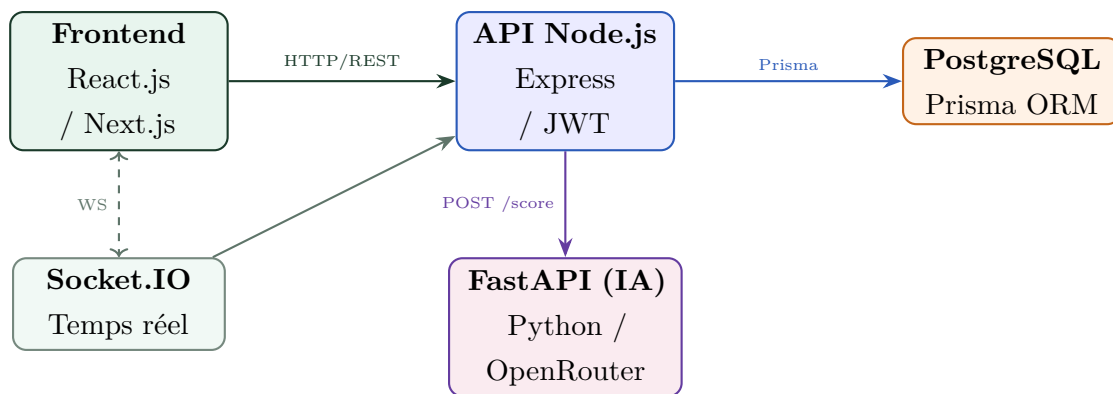


Figure 2.1 – *Architecture globale de Tanit Talent*

2.2 Stack Technologique

Table 2.1 – Stack technologique complète

Couche	Technologie	Rôle
Frontend	React.js / Next.js	Interface utilisateur, routing, SSR
Backend	Node.js / Express	Serveur HTTP, routage API REST
	Socket.IO	Communication temps réel
	JWT (cookies HttpOnly)	Authentification et sessions
	Multer	Upload de fichiers (CV, avatars)
Base de données	PostgreSQL / Prisma	Stockage et abstraction BDD
Service IA	Python / FastAPI	Microservice intelligence artificielle
	OpenRouter API	LLM pour chat et analyse de CV
Infrastructure	Docker Compose	Conteneurisation et orchestration

Chapitre 3

Diagrammes de Séquence

Convention graphique

- Les **flèches pleines** ($\longrightarrow$) représentent les **requêtes** et appels de méthode.
- Les **flèches pointillées** ($\dashrightarrow$) représentent les **réponses** et retours.
- Les **rectangles verts clairs** sur une lifeline indiquent un **traitement interne**.
- Les acteurs sont colorés par couche : **vert** = utilisateur/frontend, **bleu** = API, **orange** = base de données, **violet** = service IA.

3.1 Connexion (Login)

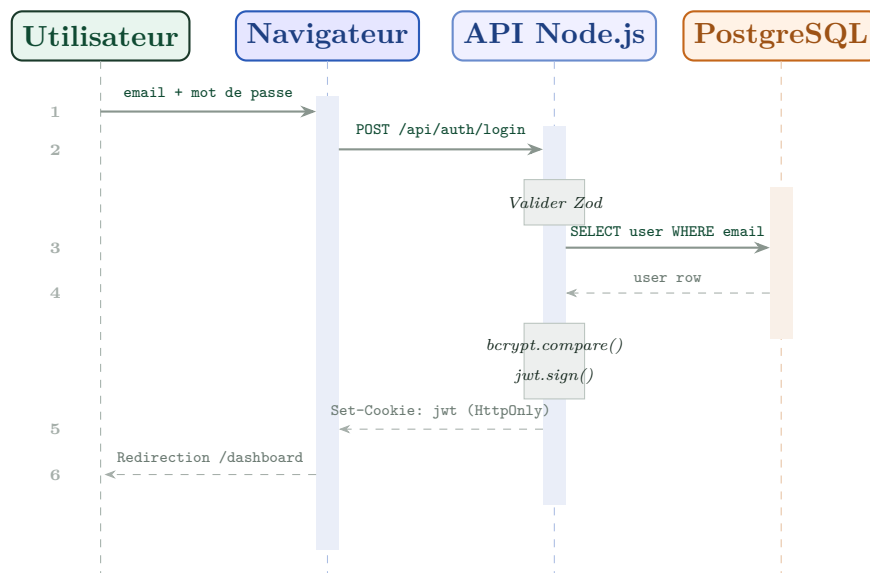


Figure 3.1 — Diagramme de séquence — Connexion

3.2 Inscription

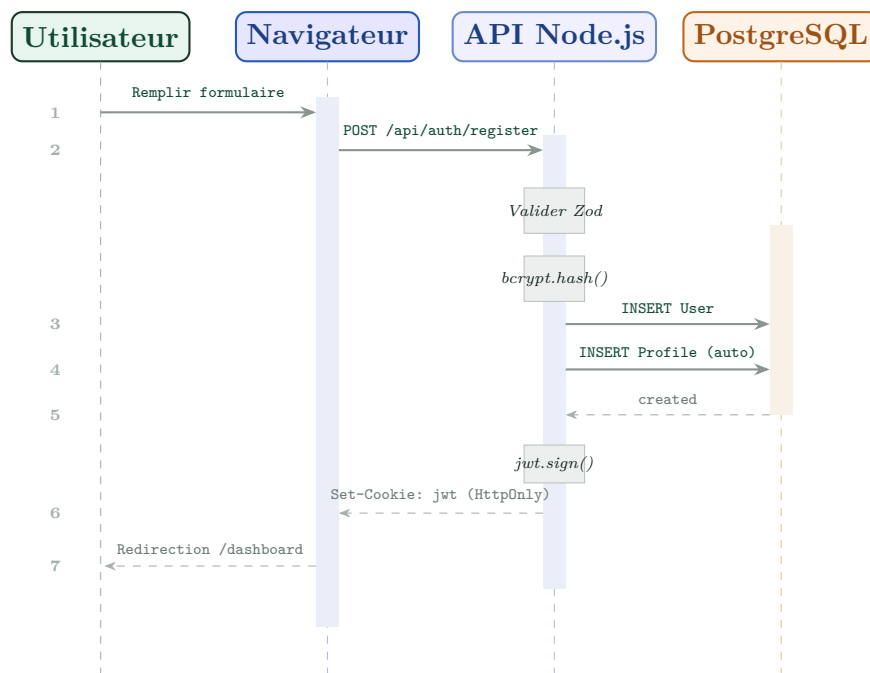


Figure 3.2 — Diagramme de séquence — Inscription

3.3 Upload CV et Analyse IA

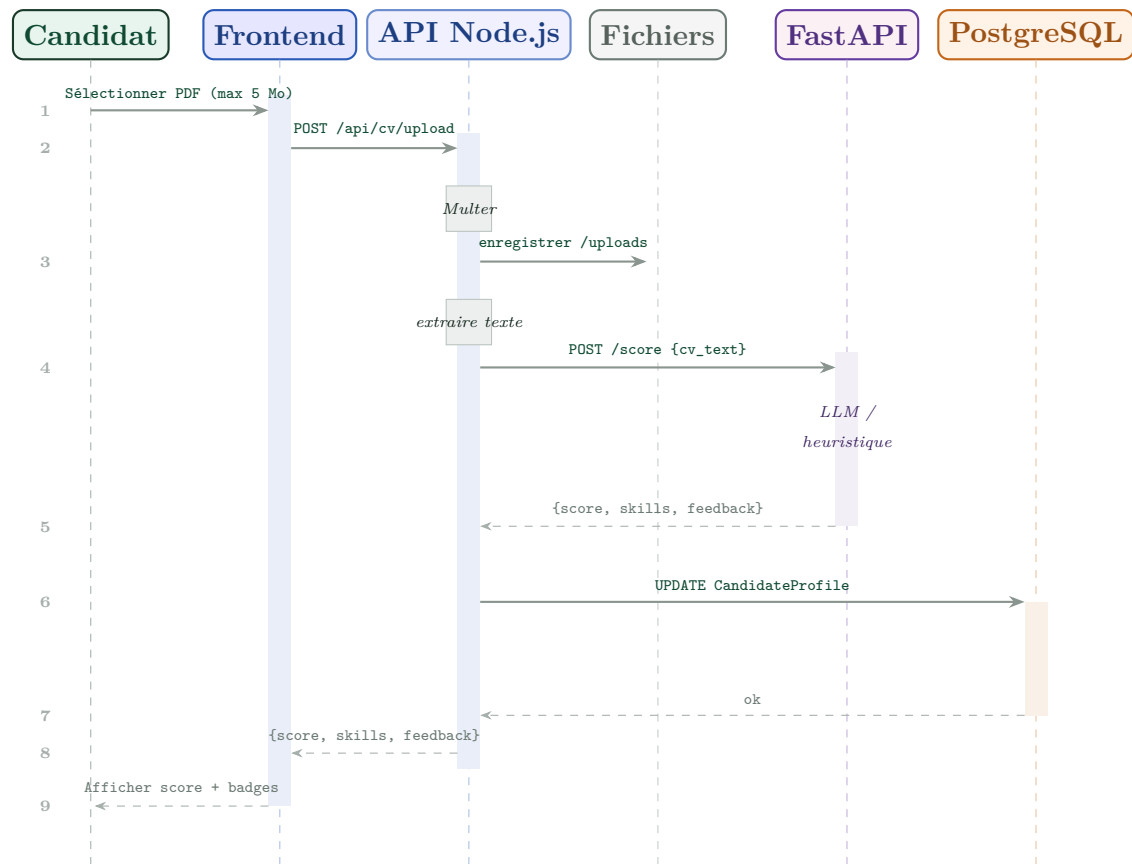


Figure 3.3 – Diagramme de séquence — Upload CV et Analyse IA

3.4 Candidature à une offre

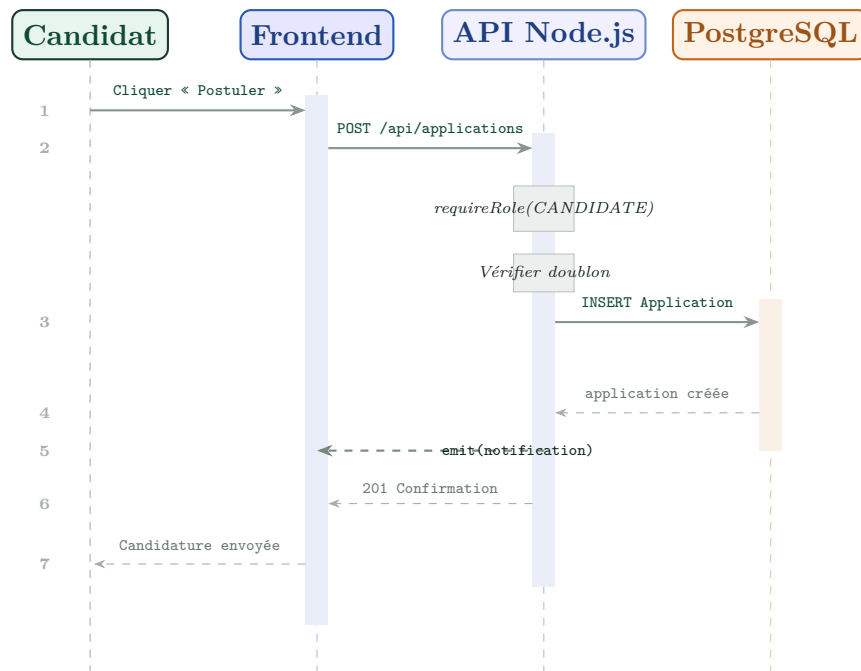


Figure 3.4 — Diagramme de séquence — Candidature à une offre

3.5 Publication d’une offre (Recruteur)

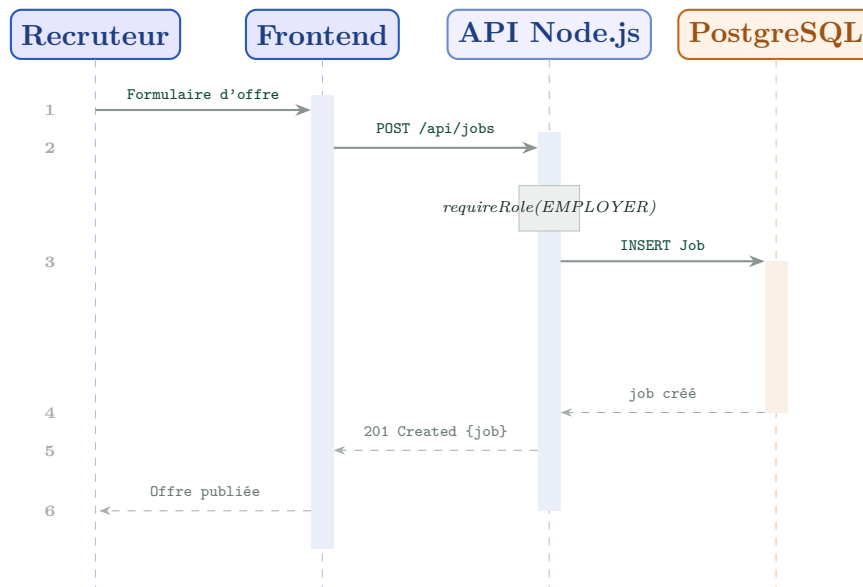


Figure 3.5 — Diagramme de séquence — Publication d’une offre

3.6 Chat avec l'assistant Tanit AI

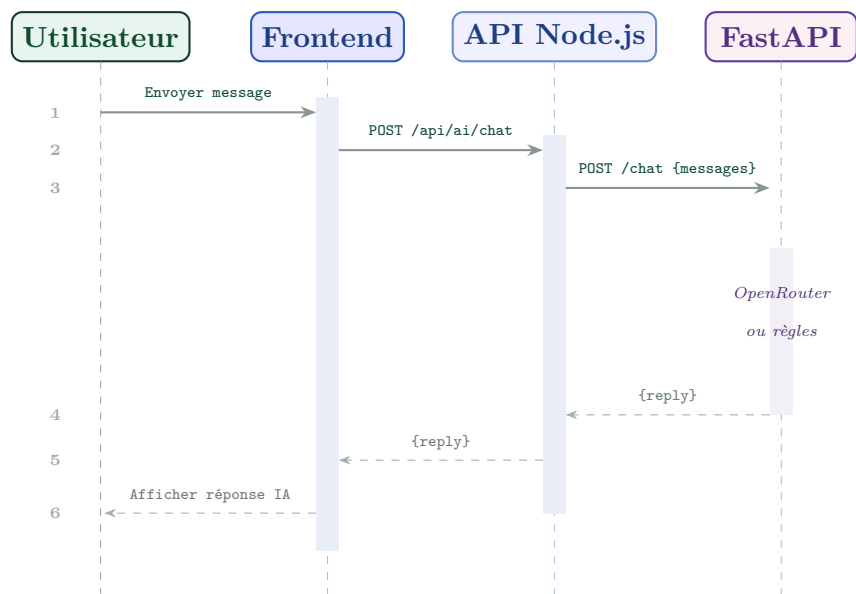


Figure 3.6 – Diagramme de séquence — Chat Tanit AI

3.7 Suggestions d'offres personnalisées

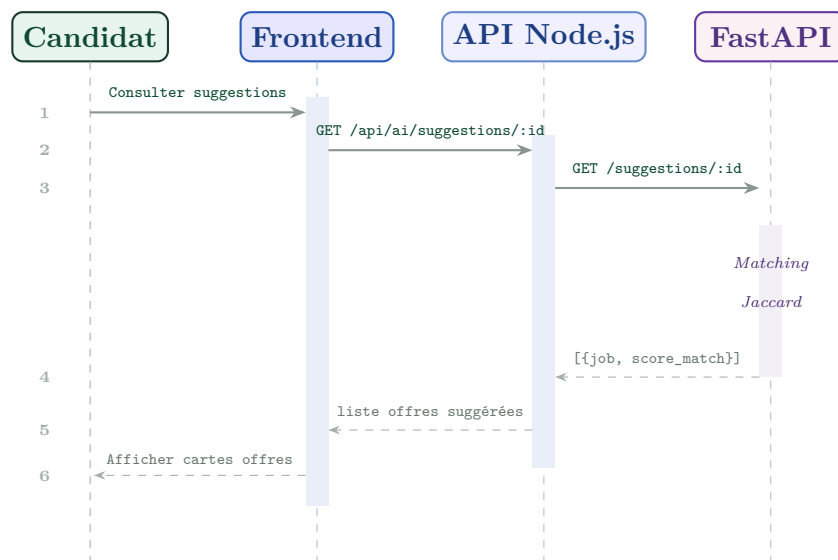


Figure 3.7 – Diagramme de séquence — Suggestions d'offres

3.8 Mise à jour du statut d'une candidature

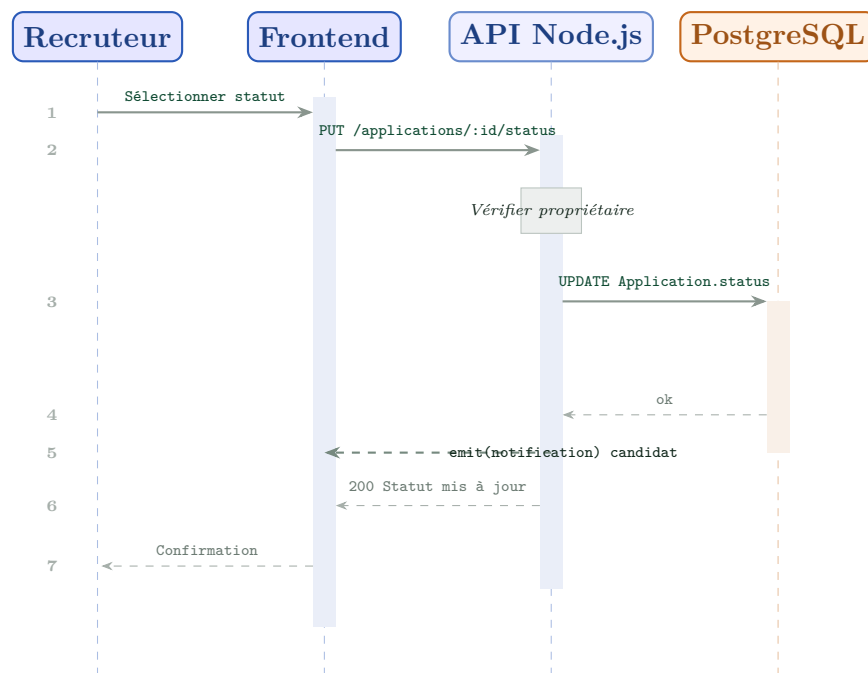


Figure 3.8 – Diagramme de séquence — Mise à jour du statut d'une candidature

3.9 Récapitulatif des flux

Table 3.1 – *Récapitulatif des diagrammes de séquence*

#	Flux	Acteurs	Points clés
1	Connexion	User, Browser, API, DB	bcrypt.compare, jwt.sign, cookie HttpOnly
2	Inscription	User, Browser, API, DB	Zod, bcrypt.hash, profil auto
3	Upload CV+IA	Candidat, Frontend, API, Fichiers, FastAPI, DB	Multer, POST /score, UPDATE profil
4	Candidature	Candidat, Frontend, API, DB	requireRole, double check, Socket.IO
5	Publication offre	Recruteur, Frontend, API, DB	requireRole(EMPLOYEE) INSERT Job
6	Chat IA	User, Frontend, API, FastAPI	Proxy /chat, OpenRouter ou fallback
7	Suggestions	Candidat, Frontend, API, FastAPI	GET /suggestions, mat-

Chapitre 4

Modèle de Données

4.1 Schéma Entité-Relation

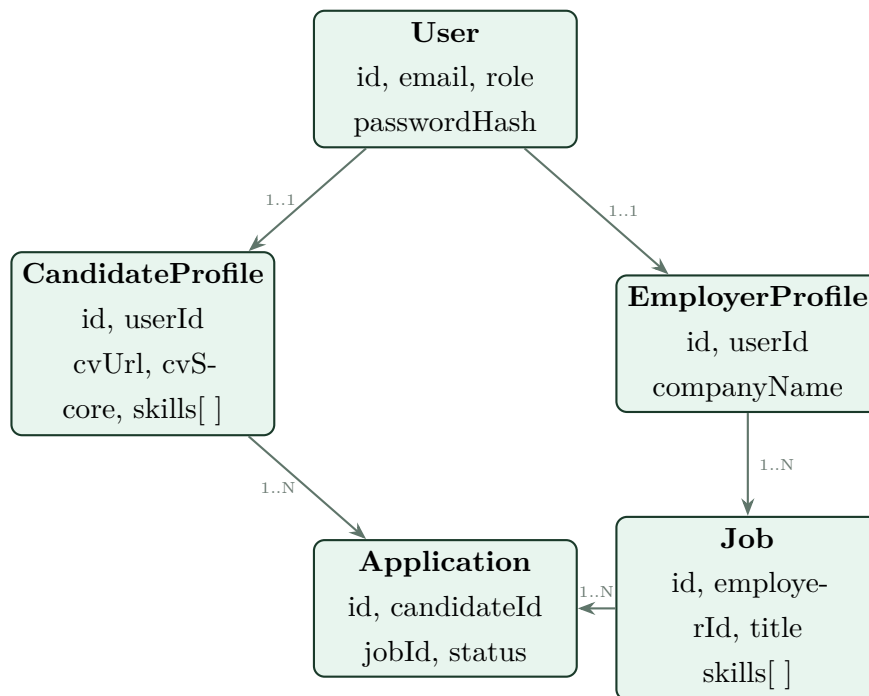


Figure 4.1 – Diagramme Entité-Relation simplifié

4.2 Schémas Prisma principaux

```
1 model User {
2   id          String          @id @default(cuid())
3   email       String          @unique
4   passwordHash String
5   role        Role            @default(CANDIDATE)
6   avatarUrl   String?
```

```
7   createdAt      DateTime      @default(now())
8   candidateProfile CandidateProfile?
9   employerProfile EmployerProfile?
10 }
11
12 model CandidateProfile {
13   id          String    @id @default(cuid())
14   userId      String    @unique
15   cvUrl       String?
16   cvScore     Float?
17   skills      String[]  @default([])
18   user        User      @relation(fields: [userId], references: [
19     id])
19   applications Application[]
20 }
21
22 enum Role { CANDIDATE EMPLOYER ADMIN }
```

Listing 4.1 — Schéma Prisma — User & CandidateProfile

Chapitre 5

API REST — Documentation

5.1 Authentification (/api/auth)

Table 5.1 – Endpoints d’authentification

Méthode	Route	Description	Auth
POST	/api/auth/register	Inscription (crée profil auto)	Non
POST	/api/auth/login	Connexion, retourne cookie JWT	Non
POST	/api/auth/logout	Déconnexion, efface cookie	Oui
GET	/api/auth/me	Utilisateur courant + profil	Oui

5.2 Offres d’emploi (/api/jobs)

Table 5.2 – Endpoints de gestion des offres

Méthode	Route	Description	Auth
GET	/api/jobs	Liste publique avec filtres	Non
GET	/api/jobs/:id	Détail d’une offre	Non
GET	/api/jobs/mine	Mes offres (recruteur)	EMPLOYER
POST	/api/jobs	Créer une offre	EMPLOYER/ADMIN
PUT	/api/jobs/:id	Modifier une offre	Propriétaire/ADMIN
DELETE	/api/jobs/:id	Supprimer une offre	Propriétaire/ADMIN

5.3 Candidatures (/api/applications)

Table 5.3 – Endpoints de gestion des candidatures

Méthode	Route	Description	Auth
POST	/api/applications	Postuler à une offre	CANDIDATE
GET	/api/applications/mine	Mes candidatures	CANDIDATE
GET	/api/applications/job/:id	Candidats d'une offre	EMPLOYER
PUT	/api/applications/:id/status	Changer statut	EMPLOYER/ADMIN

5.4 Proxy IA (/api/ai)

Table 5.4 – Endpoints proxy vers FastAPI

Méthode	Route	Description
POST	/api/ai/score	Scoring CV (proxy FastAPI)
POST	/api/ai/match	Compatibilité candidat ↔ offre
POST	/api/ai/chat	Chat avec l'assistant Tanit AI
GET	/api/ai/suggestions/:userId	Suggestions d'offres (FastAPI)
GET	/api/ai/suggestions-db/:userId	Suggestions locales (mode dégradé)

Chapitre 6

Sécurité et Authentification

6.1 Mécanisme JWT avec cookie HttpOnly

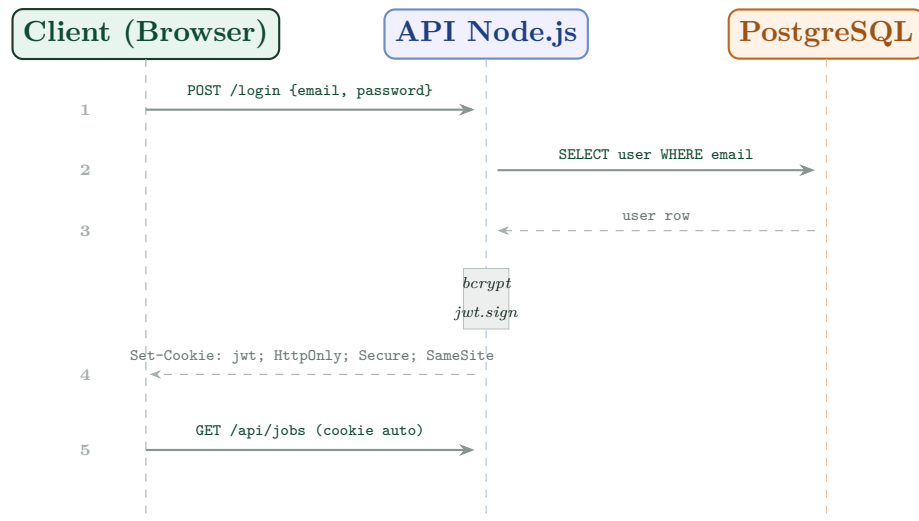


Figure 6.1 – *Flux d'authentification JWT avec cookie HttpOnly*

6.2 Mesures de Sécurité

Table 6.1 – Mesures de sécurité implémentées

Mesure	Description
Cookie HttpOnly	JWT inaccessible depuis JavaScript (protection XSS)
Cookie Secure	Transmission HTTPS uniquement en production
Cookie SameSite	Protection CSRF (Strict ou Lax)
Hachage bcrypt	Mots de passe hashés avec sel (coût ≥ 12)
Validation Zod	Validation des entrées côté API avant traitement
Rate limiting	Express-rate-limit sur les routes d'auth
Multer limits	Upload limité à 5 Mo, types MIME vérifiés
Ownership check	Vérification propriétaire avant modification

Chapitre 7

Déploiement et Infrastructure

7.1 Docker Compose

```
1 version: "3.9"
2 services:
3   postgres:
4     image: postgres:16-alpine
5     environment:
6       POSTGRES_DB:      tanit_talent
7       POSTGRES_USER:    postgres
8       POSTGRES_PASSWORD: ${DB_PASSWORD}
9     volumes: [pgdata:/var/lib/postgresql/data]
10    ports: ["5432:5432"]
11
12   api:
13     build: ./api
14     environment:
15       DATABASE_URL:      postgresql://postgres:${DB_PASSWORD}
16                          @postgres/tanit_talent
17       JWT_SECRET:        ${JWT_SECRET}
18       AI_SERVICE_URL:    http://ai:8000
19     ports: ["3001:3001"]
20     depends_on: [postgres]
21     volumes: [./uploads:/app/uploads]
22
23   ai:
24     build: ./ai-service
25     environment:
26       OPENROUTER_API_KEY: ${OPENROUTER_API_KEY}
27     ports: ["8000:8000"]
```

```

27
28   frontend:
29     build: ./frontend
30     environment:
31       NEXT_PUBLIC_API_URL: http://api:3001
32     ports: ["3000:3000"]
33     depends_on: [api]
34
35 volumes:
36   pgdata:

```

Listing 7.1 – *docker-compose.yml (simplifié)*

7.2 Variables d’Environnement

Table 7.1 – *Variables d’environnement requises*

Variable	Service	Description
DATABASE_URL	API	Chaîne de connexion PostgreSQL
JWT_SECRET	API	Clé secrète JWT (≥ 64 chars)
AI_SERVICE_URL	API	URL du microservice FastAPI
OPENROUTER_API_KEY	IA	Clé OpenRouter (optionnelle)
FRONTEND_URL	API	URL du frontend (CORS)
NODE_ENV	API	development ou production

Conclusion

Tanit Talent représente une solution complète pour le marché du recrutement en Tunisie. Les diagrammes de séquence présentés dans ce rapport illustrent précisément les interactions entre les composants de la plateforme, de l'authentification à l'analyse IA des CVs.

- **Séparation des responsabilités** : Frontend, API et service IA évoluent indépendamment.
- **Résilience** : Le mode dégradé garantit la continuité sans FastAPI.
- **Sécurité** : JWT + HttpOnly cookies, validation des entrées, gestion fine des rôles.
- **Extensibilité** : Docker permet d'ajouter de nouveaux microservices facilement.



Tanit Talent — Connecter les talents, construire l'avenir

Annexe A

Glossaire

API REST	Interface HTTP avec ressources identifiées par des URLs.
JWT	JSON Web Token — jeton d’authentification signé.
FastAPI	Framework Python hautes performances pour APIs.
Prisma	ORM TypeScript/JavaScript pour PostgreSQL.
Socket.IO	Bibliothèque JS pour la communication temps réel.
OpenRouter	Agrégateur de modèles de langage (LLM).
Jaccard	Similarité entre ensembles : $J(A, B) = A \cap B / A \cup B $.
HttpOnly	Attribut cookie empêchant l’accès depuis JavaScript.
Multer	Middleware Node.js pour l’upload de fichiers.